

1. Assignment Title

Step-by-Step Accuracy Improvement on CIFAR-10 Using Feedforward Neural Networks

2. Objective

In this assignment, you will classify images from the **CIFAR-10 dataset** using a **feedforward neural network**, also known as a **multilayer perceptron**.

The goal is to improve classification accuracy gradually by experimenting with different neural network design choices.

You will investigate the effects of:

- Number of neurons
- Number of hidden layers
- Batch normalization
- Activation functions
- Optimizers
- Loss functions

At every stage, you must train the model, plot the results, compare accuracy, and explain whether the change improved or reduced performance.

3. Dataset

Use the **CIFAR-10 dataset**.

CIFAR-10 contains 60,000 color images of size:

$32 \times 32 \times 3$

You may load the dataset using TensorFlow/Keras:

```
from tensorflow.keras.datasets import cifar10  
  
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
```

4. Important Restriction

You must use a **feedforward neural network only**.

Therefore, each image must be flattened:

$$32 \times 32 \times 3 = 3072$$

So each image becomes a vector of length 3072.

Number of Epochs should be 30.

5. Data Preprocessing

Normalize the pixel values:

```
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

Flatten the images:

```
x_train = x_train.reshape(-1, 3072)
x_test = x_test.reshape(-1, 3072)
```

Create a validation set from the training data:

```
from sklearn.model_selection import train_test_split

x_train, x_val, y_train, y_val = train_test_split(
    x_train,
    y_train,
    test_size=0.2,
    random_state=42
)
```

6. Step-by-Step Experiments

Students must improve the model slowly and systematically.

Do not directly jump to the best model.

Experiment 1: Baseline Feedforward Neural Network

Train a simple feedforward neural network.

Example architecture:

Input: 3072
Dense: 256 neurons, ReLU
Dense: 10 neurons, Softmax

Use:

Optimizer: SGD
Loss: Sparse Categorical Crossentropy
Epochs: 20
Batch size: 64

You must plot:

- Training accuracy vs validation accuracy
- Training loss vs validation loss

You must report:

- Training accuracy
- Validation accuracy
- Test accuracy
- Observation

Experiment 2: Increase the Number of Neurons

Now increase the number of neurons in the hidden layer.

Try at least three configurations:

Model A: 256 neurons
Model B: 512 neurons
Model C: 1024 neurons

Example:

```
Dense(512, activation="relu")
```

You must compare the validation accuracy and test accuracy.

Create a bar chart:

X-axis: Number of neurons
Y-axis: Test accuracy

Discussion questions:

1. Does increasing the number of neurons improve accuracy?
2. Does the model start overfitting?
3. Which configuration performs best?

Experiment 3: Increase the Number of Hidden Layers

Now change the number of hidden layers.

Try at least three models:

Model A: 1 hidden layer
Model B: 2 hidden layers
Model C: 3 hidden layers

Example architecture:

Input: 3072
Dense: 512, ReLU
Dense: 256, ReLU
Dense: 128, ReLU
Output: 10, Softmax

You must plot:

- Training accuracy vs validation accuracy
- Training loss vs validation loss

Discussion questions:

1. Does a deeper feedforward network improve accuracy?
 2. Does the model overfit faster?
 3. Which depth gives the best validation accuracy?
-

Experiment 4: Add Batch Normalization

Now add **Batch Normalization** after dense layers.

Compare two models:

Model A: Without Batch Normalization
Model B: With Batch Normalization

You must plot:

- Training accuracy vs validation accuracy
- Training loss vs validation loss
- Test accuracy comparison with and without batch normalization

Discussion questions:

1. Does batch normalization improve validation accuracy?
 2. Does batch normalization make training more stable?
 3. Does the loss curve become smoother?
-

Experiment 5: Try Different Activation Functions

Use the same architecture, but change the activation function.

Try at least four activation functions:

Sigmoid
Tanh
ReLU
Leaky ReLU

Optional:

ELU
GELU
Swish

Create a bar chart:

X-axis: Activation function
Y-axis: Test accuracy

Discussion questions:

1. Which activation function performs best?
 2. Why do sigmoid and tanh often perform worse in deeper networks?
 3. Why does ReLU usually train faster?
-

Experiment 6: Try Different Optimizers

Use the best architecture found so far.

Try at least four optimizers:

SGD
SGD with Momentum
RMSprop
Adam

Optional:

AdamW
Nadam

You must create:

- A line plot of validation accuracy for each optimizer
- A bar chart of final test accuracy for each optimizer

Discussion questions:

1. Which optimizer converges fastest?
 2. Which optimizer gives the best validation accuracy?
 3. Does faster convergence always mean better generalization?
-

Experiment 7: Try Different Loss Functions

Use the best architecture and optimizer found so far.

Try at least three loss functions:

Sparse Categorical Crossentropy
Categorical Crossentropy
Mean Squared Error

Optional:

Categorical Crossentropy with Label Smoothing
Focal Loss

For categorical crossentropy, convert labels into one-hot vectors:

```
from tensorflow.keras.utils import to_categorical
```

```
y_train_cat = to_categorical(y_train, 10)  
y_val_cat = to_categorical(y_val, 10)  
y_test_cat = to_categorical(y_test, 10)
```

You must compare:

- Training accuracy
- Validation accuracy
- Test accuracy
- Training stability

Create a bar chart:

X-axis: Loss function

Y-axis: Test accuracy

Discussion questions:

1. Which loss function performs best?
2. Which loss function gives the most stable training curve?